

Deep Learning and Its Application

Week 5: Practicals

1. Given the following fragment code

```
from sklearn.datasets import make_classification
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import SGD
import numpy as np

# create the dataset
X, y = make_classification(n_samples=1000, n_features=4, n_classes=2,
random_state=1)
# determine the number of input features
n_features = X.shape[1]
# define model

.
.
.
```

Follow 5 steps introduced in Lecture 3 and use the Sequential model to develop a binary classification model with the following tasks:

- Construct 1 hidden layer, consisting of 10 neurons with “relu” activation function
- Add the final layer with “sigmoid” activation function (think about why?)
- Add optimizer with “SGD” and loss function with “binary_crossentropy”
- Save the trained model as “model”
- Load the model. Given an array [1.91518414, 1.14995454, -1.52847073, 0.79430654], use the loaded model to predict the result

Answers

```
from sklearn.datasets import make_classification
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import SGD
import numpy as np

# create the dataset
X, y = make_classification(n_samples=1000, n_features=4, n_classes=2,
random_state=1)
# determine the number of input features
n_features = X.shape[1]
# define model
model = Sequential()
model.add(Dense(10, activation='relu', kernel_initializer='he_normal',
input_shape=(n_features,)))
model.add(Dense(1, activation='sigmoid'))
# compile the model
sgd = SGD(learning_rate=0.001, momentum=0.8)
model.compile(optimizer=sgd, loss='binary_crossentropy')
# fit the model
model.fit(X, y, epochs=100, batch_size=32, verbose=0, validation_split=0.3)
# save model to file
model.save('model.h5')
```

```

from tensorflow.keras.models import load_model

# load the model from file
model = load_model('model.h5')
# make a prediction
row = [1.91518414, 1.14995454, -1.52847073, 0.79430654]
row2 = np.array([[1.91518414, 1.14995454, -1.52847073, 0.79430654]])
yhat = model.predict(row2)
print('Predicted: %.3f' % yhat[0])

```

2. Given the following fragment code

```

from numpy import argmax
from pandas import read_csv
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
import numpy as np

# load the dataset
path = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/iris.csv'
df = read_csv(path, header=None)
# split into input and output columns
X, y = df.values[:, :-1], df.values[:, -1]
# ensure all data are floating point values
X = X.astype('float32')
# encode strings to integer
y = LabelEncoder().fit_transform(y)
# split into train and test datasets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
# determine the number of input features
n_features = X_train.shape[1]
.
.
.

```

Follow 5 steps introduced in Lecture 3 and use the Sequential model to develop a multiclass classification model with the following tasks:

- f) Construct 3 hidden layers, the first hidden layer consists of 20 neurons with “relu” activation function, the second composed of 10 neurons with “tanh” activation function, the third composed of 8 neurons with “sigmoid” activation function.
- g) The final layer is with “softmax” activation function (think about why?)
- h) Save the trained model
- i) Load the model. Given an array [5.1,3.5,1.4,0.2], use the loaded model to predict its result

Answer:

```
from numpy import argmax
from pandas import read_csv
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
import numpy as np

# load the dataset
path = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/iris.csv'
df = read_csv(path, header=None)
# split into input and output columns
X, y = df.values[:, :-1], df.values[:, -1]
# ensure all data are floating point values
X = X.astype('float32')
# encode strings to integer
y = LabelEncoder().fit_transform(y)
# split into train and test datasets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
# determine the number of input features
n_features = X_train.shape[1]
# define model
model = Sequential()
model.add(Dense(20, activation='relu', kernel_initializer='he_normal',
input_shape=(n_features,)))
model.add(Dense(10, activation='tanh', kernel_initializer='he_normal'))
model.add(Dense(8, activation='sigmoid', kernel_initializer='he_normal'))
model.add(Dense(3, activation='softmax'))
# compile the model
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
# fit the model
model.fit(X_train, y_train, epochs=150, batch_size=32, verbose=0)
# evaluate the model
loss, acc = model.evaluate(X_test, y_test, verbose=0)
print('Test Accuracy: %.3f' % acc)

model.save('model2.h5')

from tensorflow.keras.models import load_model
# load the model from file
model = load_model('model2.h5')

# make a prediction
row1 = [5.1,3.5,1.4,0.2]

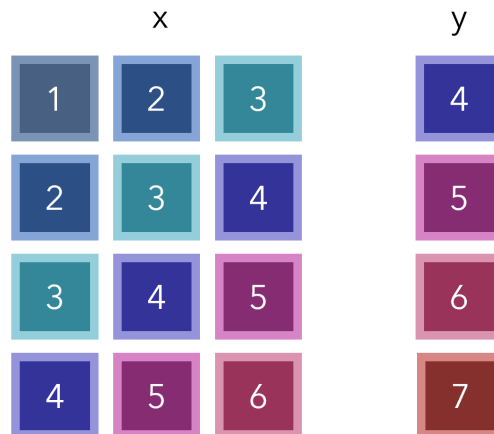
row2 = np.array([[5.1,3.5,1.4,0.2]])

yhat = model.predict([row2])
print('Predicted: %s (class=%d)' % (yhat, argmax(yhat)))
```

3. Consider the following sequence



We want to generate the following set of sequences as x and predict y values, in this case the window size is 3



Use the Sequential model to develop a simple RNN neural network with the following tasks

- Construct one hidden layer with “SimpleRNN” composed of 10 neurons
- The output layer is with “Linear” activation function
- Add optimizer with “Adm”, loss function with “mse”, and metrics with “mae”
- Run your model with epochs = 10, 50, 500
- Evaluate the model with the training set of sequences
- Given an input sequence [2, 3, 4], make predictions

Here is a piece of fragment code:

```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense
import numpy as np
from numpy import asarray
from numpy import sqrt

train = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
windowSize, X_train, y_train = 3, [], []
for index in range(len(train)-windowSize):
    X_train.append(train[index:index+windowSize])
    y_train.append(train[index+windowSize])

X_train, y_train = np.array(X_train), np.array(y_train)
X_train = X_train.reshape((len(X_train), 3, 1))

model =
```

Answer:

```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense
import numpy as np
from numpy import asarray
from numpy import sqrt

train = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
windowSize, X_train, y_train = 3, [], [] # set window size and initialize lists
for index in range(len(train)-windowSize):
    X_train.append(train[index:index+windowSize])
    y_train.append(train[index+windowSize])

X_train, y_train = np.array(X_train), np.array(y_train)
X_train = X_train.reshape((len(X_train), 3, 1)) #reshape X_train to proper 3-d
array

model = Sequential()
model.add(SimpleRNN(10, input_shape=(3,1)))
model.add(Dense(1,activation='linear'))

model.compile(loss='mse', optimizer='Adam', metrics=['mae'])
model.fit(X_train,y_train, epochs=500)
#history = model.fit(X_train,y_train,epochs=50)

mse, mae = model.evaluate(X_train, y_train, verbose=0)
print('MSE: %.3f, RMSE: %.3f, MAE: %.3f' % (mse, sqrt(mse), mae))

row = np.array([[2, 3, 4]])
row1 = row.reshape(1, windowSize, 1)
yhat = model.predict([row1])
row2 = asarray([2, 3, 4]).reshape((1, windowSize, 1))
yhat = model.predict(row2)

print(yhat)
```